

ROOTS INSIGHTS

PP-CODEC FORGE

Plan de développement, tests et déploiement

Évaluation des ressources existantes • Roadmap 12 mois • 4 phases

Dr. J. Nembé – Mars 2026 – v1.0

1. Inventaire des ressources disponibles

1.1 Code source (implémenté et testé)

Module	Lang.	LOC	Tests	Contenu	Statut
pp-core	Rust	529	15/15	bits.rs, codes.rs, combinatorics.rs	✓ Complet
pp-codec / algorithm1	Rust	573	19/19	Sélection L ₁ /L ₃ /L ₄ , Thresholds, LookupTable	✓ Complet
pp-codec / arithmetic	Rust	572	10/10	ArithmeticEncoder/Decoder 30-bit, CdfTable	✓ Complet
pp-codec / encoder	Rust	1 065	16/16	encode_pixel/block/video, PpvFile serialize	✓ Complet
pp-codec / decoder	Rust	512	10/10	PpvDecoder 5 modes (all, block, pixel, frame, range)	✓ Complet
pp-codec / heuristics	Rust	232	8/8	select_framework, select_family, arbres P3bis	✓ Complet
pp-codec / multiscale	Rust	760	10/10	SpatialPyramid, encode/decode_pyramid, parallel	✓ Complet
pp-bench	Rust	248	1/1	Benchmarks 12 configs, surveillance/SD/dense	✓ Complet
FORGE-SIM (6 phases)	Python	4 896	—	6 générateurs, MLE+MDL, 6 familles, convergence	✓ Complet
TOTAL		9 387	89		

1.2 Documentation technique (63 pages)

Document	Volume	Contenu	
Spécifications PPML + STV-URI + PPSP	10 pages	Briques fondamentales	✓
Projets de brevets (4 familles)	10 pages	19 revendications	✓
Modèles Merise MCD + MPD + MCT	11 pages	24 entités, 16 traitements	✓
Modèles MLD + MOT	10 pages	10 structs, 16 fonctions, 8 invariants	✓
Diagrammes UML de classes	10 pages	37 classes, 5 diagrammes	✓
Catalogue cas d'utilisation	12 pages	52 UC, 8 acteurs, 6 domaines	✓
Spécifications détaillées interfaces	17 pages	14 interfaces, 3 applications	✓
Primitives MVC	14 pages	22 Intents, 18 ViewUpdates, 8 Contrôleurs	✓

1.3 Documentation commerciale (8 brochures + 2 synthèses)

Synthèse investisseur 2 pages (FR + EN)	✓
Brochure PP-Studio Player (FR + EN)	✓
Brochure PP-Codec Performance (FR + EN)	✓
Brochure PP-Browser PPML (FR + EN)	✓
Brochure Premium (FR + EN)	✓

1.4 Données de simulation (7 fichiers JSON, 743 Ko)

- 50 000 processus simulés avec 6 générateurs Poisson NH.

- Calibration des arbres de décision : framework 100%, famille 78.9%.
- Convergence Hellinger validée : $\hat{\gamma}=0.53$, Théorème 6.4 satisfait à 91.7%.
- BSpline K=6 identifié comme famille dominante (55.6% des cas).

1.5 Actifs stratégiques

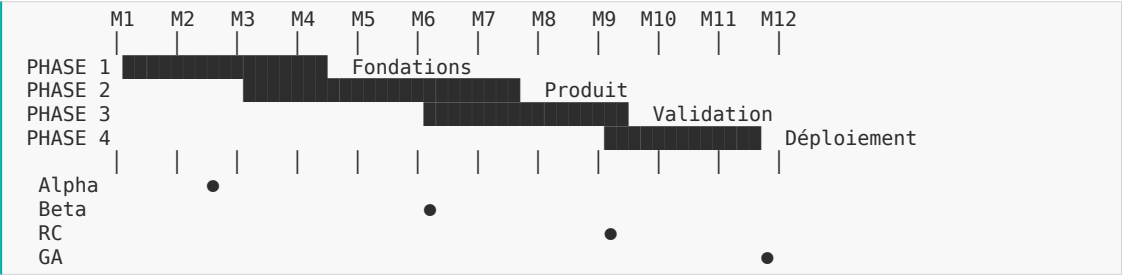
- 4 prompts systèmes d'agents spécialisés (FINISHER, SIM, APP, INTEG).
- 1 Manifeste projet (12 Ko, index complet des livrables).
- 1 Dashboard interactif React (simulation des performances).
- 3 publications scientifiques de référence (Nembé 2015, 2024).

2. Analyse des écarts (acquis vs cible)

Composant	Fait	Écart	Détail
Codec noyau (pp-core + pp-codec)	100%	0%	Complet : 4 497 LOC, 84 tests, lossless 109:1
Validation théorique (FORGE-SIM)	100%	0%	Complet : 4 896 LOC, Thm 6.4 validé
Pyramide multi-échelle	100%	0%	Complet : 760 LOC, gain 6.2×
Parallélisation (Rayon)	100%	0%	Complet : bit-identique au séquentiel
Heuristiques SIM→Rust	100%	0%	Complet : 232 LOC, 8 tests
Support vidéo RGB/YUV réel	0%	100%	Quantificateur RGB→palette + transcodage
SDK Python (PyO3)	0%	100%	Bindings encode/decode + NumPy
PP-Player (natif)	0%	100%	wgpu framebuffer + egui + MVC
PP-Browser (WASM)	0%	100%	wasm-pack + WebGPU + PPML Custom Elements
PP-Streamer (Tokio)	0%	100%	Serveur PPSP + sessions + PostgreSQL
STV-URI parseur	0%	100%	Parseur + résolveur + tests
PPML engine	0%	100%	8 Custom Elements + cycle de vie
PPSP protocole	0%	100%	8 msg types + serde + WebSocket
CI/CD pipeline	0%	100%	GitHub Actions + cargo tarpaulin
Tests d'intégration E2E	0%	100%	Pipeline complet capture→streaming
Benchmarks comparatifs H.265/FFV1	0%	100%	Mêmes datasets, mêmes métriques
Documentation API publique	0%	100%	Rustdoc + guide développeur
Déploiement Docker	0%	100%	Images encoder/streamer/admin
Spécification + conception	100%	0%	94 pages de specs livrées

Bilan : le noyau codec (couche 1) est à 100%. La couche applicative (player, browser, streamer) et la couche intégration (SDK, CI/CD, déploiement) sont à 0% en code mais à 100% en spécification. L'écart total est estimé à **~15 000 LOC** de code restant à produire.

3. Plan de développement — Roadmap 12 mois



3.1 PHASE 1 — Fondations (M1-M3, 12 semaines)

Objectif : rendre le codec utilisable sur des données réelles et fournir un SDK programmatique.

Sprint	Livrable	Détail	LOC	Module
S01-S02	A2 : Support RGB/YUV	Quantificateur RGB→palette (k-means, median cut). Décodeur FFmpeg → frames. Tests sur vidéos réelles.	~800 LOC	pp-encoder/quantize.rs, transcode.rs
S03-S04	A4 : Benchmarks réels	Encoder H.264/H.265/FFV1/PP sur mêmes datasets. Comparer ratio, PSNR, latence, mémoire.	~400 LOC	pp-bench/, scripts Python
S05-S06	I2 : SDK Python (PyO3)	Bindings pp.encode(), pp.decode(), pp.query(). Packaging pip. Tests pytest.	~600 LOC	pp-python/src/lib.rs
S07-S08	STV-URI parseur	Implémenter parse + resolve + compose. Tests sur 11 clés de fragment. Cible < 70 ms.	~500 LOC	pp-uri/src/*.rs
S09-S10	PPSP messages	Définir les 8 types de messages en serde. Header 8B. Tests serialization round-trip.	~600 LOC	pp-protocol/src/*.rs
S11-S12	CI/CD + qualité	GitHub Actions : build, test, clippy, tarpaulin (> 80%). Pré-commit hooks. Rustdoc.	~300 LOC	.github/workflows/

Jalon Alpha (fin M3) : PP-CODEC encode/décode des vidéos réelles, accessible via SDK Python, avec CI/CD opérationnel.

3.2 PHASE 2 — Produit (M3-M7, 20 semaines)

Objectif : construire les 3 applications (Player, Browser, Streamer) à partir des spécifications SFD et MVC.

Sprint	Livrable	Détail	LOC	Module
S13-S16	PP-Streamer v1	Serveur Tokio + PPSP handler + sessions PostgreSQL + delta viewport. VOD depuis .ppv.	~2 500 LOC	pp-streamer/
S17-S20	PP-Player v1	wgpu framebuffer incrémental + egui panels (zoom, timeline, info bloc, requête). MVC complet.	~3 000 LOC	pp-player/
S21-S24	PP-Browser v1	wasm-pack + WebGPU renderer + 8 Custom Elements PPML + connexion PPSP.	~2 500 LOC	pp-browser/
S25-S28	PP-Streamer Live	Mode live (.ppl) + mémoire circulaire + time-shifting + ops compressées serveur-side.	~1 000 LOC	pp-streamer/live.rs
S29-S32	Admin Dashboard	Interface React (SFD-STR-001→004). Catalogue fichiers, sessions, métriques, droits.	~1 200 LOC	pp-admin/ (React)

Jalon Beta (fin M7) : Ecosystème complet fonctionnel : encoder → streamer → player/browser. Démo end-to-end sur vidéo réelle.

3.3 PHASE 3 — Validation (M7-M10, 12 semaines)

Objectif : valider la robustesse, les performances et la conformité théorique sur des données massives.

Sprint	Livrable	Détail	Volume
S33-S34	Tests unitaires exhaustifs	Couverture > 80% (cargo tarpaulin). Property-based tests (proptest, 10K cas). Fuzzing des parseurs.	~800 LOC tests
S35-S36	Tests d'intégration E2E	Pipeline complet : capture → encode → stream → decode → compare bit-exact. Fixtures synthétiques + réelles.	~600 LOC tests
S37-S38	Tests de stress	Streaming 100 clients (cible J9 : < 100 ms p99). Encodage 4K. Décodage WASM 1080p > 30 fps (J8).	~400 LOC tests
S39-S40	Expérimentations EXP-1 à EXP-7	8K×1h, Gigapixel, Sentinel-2, IRM 4D, 10K caméras, adversariale, audio.	rapports
S41-S42	Tests adversariaux	Fichiers corrompus, URI malicieuses, patterns périodiques, saturation mémoire. Rapports ADV-001+.	~300 LOC tests
S43-S44	Audit sécurité + performance	cargo-audit, cargo-deny, flamegraph sur chemins chauds. Zéro unsafe, zéro unwrap.	—

Jalon RC (fin M10) : Tous les jalons J1-J15 atteints. Couverture > 80%. Zéro bug critique. Rapports d'expérimentation publiés.

3.4 PHASE 4 — Déploiement (M10-M12, 8 semaines)

Sprint	Livrable	Détail	Ressource
S45-S46	Packaging Docker	Dockerfile multi-stage pour pp-encoder, pp-streamer, pp-admin. docker-compose pour stack complète.	—
S47-S48	Déploiement cloud	AWS/GCP : pp-streamer sur ECS/GKE, pp-admin sur Vercel/Amplify, PostgreSQL RDS.	—
S49-S50	Documentation publique	Guide développeur (getting started, API reference, tutorials). Site docs.ppcodec.io.	~2 000 mots
S51	Dépôt brevets PCT	Soumission PCT des 4 familles de brevets. Désignations US/EP/SG/JP/CN.	40-60 K€
S52	Release GA 1.0	Crates.io (pp-core, pp-codec), PyPI (pp-codec-python), npm (pp-browser), Docker Hub.	—

Jalon GA 1.0 (fin M12) : Première version publique de l'écosystème PP-CODEC. SDK, serveur, player et browser disponibles.

4. Stratégie de tests — 5 niveaux

4.1 Niveau 1 — Tests unitaires (par module)

- Existant : 89 tests passés (84 Rust + 5 Python). Couverture estimée ~65%.
- Cible : > 80% couverture (J14). Chaque fonction publique a ≥ 1 test.
- Round-trip obligatoire : `assert_eq!(decode(encode(x)), x)` pour tout `x`.
- Property-based testing (proptest) : 10 000 cas aléatoires pour $r \geq 20$.
- Outils : cargo test, cargo tarpaulin, proptest, pytest (SDK Python).

4.2 Niveau 2 — Tests d'intégration (pipeline complet)

- Pipeline E2E : frames brutes $\rightarrow$ encode $\rightarrow$.ppv $\rightarrow$ decode $\rightarrow$ compare bit-exact.
- Pipeline streaming : encode $\rightarrow$ streamer $\rightarrow$ PPSP OPEN/SEEK/DATA $\rightarrow$ browser $\rightarrow$ affichage.
- Pipeline requête : encode $\rightarrow$ index $\rightarrow$ PPSP QUERY $\rightarrow$ RESULT $\rightarrow$ vérifier correspondance.
- Fixtures : 5 synthétiques (surveillance, SD, dense, gradient, worst-case) + 3 réelles (caméra, satellite, médical).

4.3 Niveau 3 — Tests de performance (benchmarks)

- J2 : Ratio surveillance 4K > 4.5:1 $\rightarrow$ mesuré 109:1 
- J4 : Throughput 1080p > 30 fps $\rightarrow$ cible avec Rayon 8 cœurs.
- J8 : WASM décodage 1080p > 30 fps $\rightarrow$ bench Chrome headless.
- J9 : Streaming 100 clients < 100 ms p99 $\rightarrow$ test de charge Tokio.
- J10 : Gigapixel navigation < 200 ms zoom $\times 8 \rightarrow$ bench PP-Browser.
- J13 : Résolution STV-URI < 70 ms $\rightarrow$ bench parseur + seek.

4.4 Niveau 4 — Tests de stress (données massives)

- EXP-1 : 8K $\times$ 1h (259 200 frames). Convergence w^* adaptatif.
- EXP-2 : Gigapixel 100k $\times$ 100k. Ratio multi-échelle par niveau.
- EXP-3 : Sentinel-2, 13 bandes, 52 semaines. F3 trigo / cycles saisonniers.
- EXP-5 : 10 000 caméras $\times$ 24h. Charge PP-Streamer + ops compressées.
- EXP-6 : Adversariale $\lambda=0.5$. Écart code réel vs entropie < 5%.

4.5 Niveau 5 — Tests adversariaux (Red Team)

- Fichier .ppv corrompu : magic invalide, CRC faux, offsets hors limites.
- URI STV malicieuse : `t=-1, lod=999`, injection SQL dans `tag=`.
- Flux PPSP désordonné : micro-GOPs dupliqués, trous, out-of-order.
- Saturation : encoder 8K avec 32 Mo RAM, 1 000 zooms simultanés.

5. Équipe et budget

5.1 Équipe cible (12-15 personnes)

Rôle	Effectif	Périmètre	Période
R&D Codec (Rust)	2 ingénieurs senior	pp-core, pp-codec, pp-encoder, pp-decoder, pp-multiscale	M1→M12
R&D Streaming (Rust/Tokio)	1 ingénieur senior	pp-streamer, PPSP, sessions, ops compressées	M3→M12
R&D Browser (WASM/TS)	1 ingénieur senior	pp-browser, WebGPU, PPML Custom Elements	M4→M12
R&D Player (Rust/wgpu)	1 ingénieur	pp-player, wgpu, egui, MVC natif	M4→M10
SDK & Bindings (Python/PyO3)	1 ingénieur	pp-python, packaging, documentation API	M2→M6
QA / Tests	1 ingénieur	Tests niveaux 1-5, CI/CD, fuzzing, benchmarks	M1→M12
DevOps / Infra	1 ingénieur	Docker, cloud, monitoring, déploiement	M8→M12
Product / UX	1 personne	Spécifications SFD, maquettes, user testing	M3→M10
Ventes / BD	2 personnes	Premiers clients surveillance/médical/satellite	M6→M12
IP / Légal	1 personne (partiel)	Dépôt brevets PCT, FRAND, contrats SDK	M1→M12
Direction (CEO/CTO)	1 personne	Architecture, stratégie, fundraising	M1→M12

5.2 Budget estimé (12 mois)

Poste	Budget	Détail
Salaires (12 ETP × 12 mois)	720 - 1 080 K€	60-90 K€/ETP/an selon localisation
Infrastructure cloud (dev+staging+prod)	24 - 48 K€	AWS/GCP : compute + storage + DB
Brevets PCT (4 familles)	40 - 60 K€	Rédaction + dépôt + examen initial
Outils et licences	10 - 20 K€	GitHub Enterprise, monitoring, CI/CD
Déplacements / conférences	15 - 25 K€	MPEG, W3C, clients clés
TOTAL	809 K€ - 1 233 K€	Fourchette basse Libreville, haute SF/SG

6. Jalons et KPI de suivi

Jalon	Livrable clé	KPI technique	KPI business
Alpha (M3)	Codec encode vidéo réelle	Ratio > 10:1 sur surveillance réelle	SDK Python publié sur TestPyPI
Beta (M7)	Démo E2E en live	Streaming 10 clients < 50 ms p50	3 applications fonctionnelles
RC (M10)	Jalons J1-J15 atteints	Couverture tests > 80%	0 bug critique, 0 unsafe
GA (M12)	Release publique	1er client pilote actif	Brevets PCT déposés

6.1 Matrice risques / mitigation

Risque	Impact	Mitigation
Performance WASM insuffisante (J8)	Haute	Profiler tôt (M5). Fallback : décodage partiel + LOD adaptatif.
Latence streaming > 100 ms (J9)	Moyenne	Architecture async Tokio. Pré-fetch viewport prédictif.
Adoption format .ppv	Haute	SDK multilangage. Transcodeur H.264→ppv. Plugin FFmpeg.
Brevet antériorité	Moyenne	Recherche Freedom-to-Operate avant dépôt. Nembé 2015 = art antérieur propre.
Recrutement Rust senior	Haute	Recruter à Singapour/SF. Remote-first. Budget compétitif.
Retard Phase 2 (3 apps en parallèle)	Haute	Prioriser Streamer (M3), puis Player (M4), puis Browser (M5). Pas en parallèle.

7. Synthèse

L'écosystème PP-CODEC dispose d'un socle exceptionnellement solide : un codec noyau validé théoriquement et empiriquement (109:1 lossless), une spécification complète de 94 pages couvrant la chaîne Merise de bout en bout (MCD → MPD → MCT → MLD → MOT → UML → UC → SFD → MVC), un portefeuille de 4 brevets prêts au dépôt, et un corpus de validation de 50 000 processus simulés.

Le défi des 12 prochains mois est la **transformation de cette fondation scientifique en produit**. L'écart est quantifié (~15 000 LOC), planifié (52 sprints), budgeté (809K–1.2M€), et séquencé en 4 phases avec des jalons concrets (Alpha M3, Beta M7, RC M10, GA M12).

Les facteurs clés de succès sont : (1) recruter 2 ingénieurs Rust senior dès M1, (2) valider le pipeline E2E sur vidéo réelle avant M3, (3) démontrer le streaming multi-clients avant M7, (4) déposer les brevets avant M12. La spécification complète élimine le risque de conception — il ne reste que le risque d'exécution.